

Ri-acquisizione del corpus Dataverse — Runbook del pilota (Fase 1)

Direzione ICT, Università degli Studi di Milano — Progetto Arkive

Luglio 2026

Indice

Runbook del pilota — Fase 1	1
1. Scopo e contesto	1
2. Esito della Fase 0 (dati di partenza)	2
3. Dataset pilota selezionati	2
4. Prerequisiti (da verificare PRIMA di iniziare)	2
5. Esecuzione	3
5.1 Camera pulita — DENTRO la Transfer Source Location	3
5.2 Lista DOI del pilota	4
5.3 Download	4
6. Gate 1 — Validazione dell’acquisizione	4
7. Ingest e Gate 2 — Validazione dell’AIP	6
7.1 Ingest	6
7.2 Gate 2 — controlli sull’AIP	6
8. I 6 DOI non più raggiungibili	7
9. Criteri di GO / NO-GO per il lotto completo	8
10. Registrazione dell’esito	8
10.1 Esito del pilota del 22 luglio 2026	8

Runbook del pilota — Fase 1

1. Scopo e contesto

Il corpus Dataverse è stato acquisito e ingerito in Archivematica **prima** che la pipeline introducesse due comportamenti oggi ritenuti necessari:

1. la **verifica di integrità** dei file scaricati (checksum/dimensione), che impedisce a un download troncato di entrare silenziosamente nel SIP;
2. la **conservazione dell’originale** per i file ingeriti come tabellari (`format=original`), invece della derivata `.tab` generata da Dataverse.

Poiché gli AIP prodotti finora sono **sperimentali** (progetto in fase avanzata ma non ancora in produzione), è stato deciso di **non migrare** i pacchetti esistenti ma di **ri-acquisire l’intero corpus** con la pipeline corretta e ricostruire gli AIP da una baseline pulita. Questo evita di trascinare nella futura produzione catene di supersede e record di deaccession relativi a materiale di prova, e — cosa più importante — sottopone a verifica di integrità **tutto** il corpus, non solo i dataset con l’anomalia nota.

Questo documento descrive la **Fase 1**: l’esecuzione controllata del ciclo completo su **due dataset pilota**, allo scopo di validare la procedura prima di applicarla ai 702 dataset dello scope. Il prodotto della Fase 1 non è solo “due pacchetti corretti”, ma **la procedura verificata** da ripetere sul lotto.

2. Esito della Fase 0 (dati di partenza)

La ricognizione con `verifica_raggiungibilita.py` sui 708 DOI dello scope ha dato:

Indicatore	Valore
DOI ri-acquisibili (OK)	672
DOI vuoti (nessun file in nessuna versione)	30
DOI non più raggiungibili su Dataverse	6
DOI con file ad accesso ristretto	111
Versioni totali da scaricare	1 226
File totali da scaricare	67 730
Volume stimato (originali, tutte le versioni)	62,95 GB

I 6 DOI non più raggiungibili — QA4BR0, BX8AIN, YF2NZV, WKMYTV, YIAURF, OYDM83 — sono **fuori dallo scope della ri-acquisizione** e vanno trattati a mano: vedi sezione 8.

3. Dataset pilota selezionati

I due pilota sono scelti per **esercitare i percorsi nuovi**, non per comodità:

Pilota	DOI	Versioni	File	Ingeriti	Ristretti	Volume
A	doi:10.13130/RD_UNIMI/PKZ7FJ	4	10	8	0	4,3 MB
B	doi:10.13130/RD_UNIMI/DP08SC	3	26	23	25	1,1 MB

Perché A: quattro versioni e otto file ingeriti su dieci. Valida in un colpo solo la struttura per versione (`objects/<versione>/objects/`), il download `format=original` e l'identificazione dei formati reali, restando piccolo.

Perché B: quasi tutti i file sono **contemporaneamente ristretti e ingeriti** (25 ristretti, 23 ingeriti su 26). È l'intersezione più rischiosa dell'intero corpus: richiede che `format=original` funzioni **su file ad accesso ristretto con API key**. Se in quella combinazione mancasse un permesso, il pacchetto risulterebbe silenziosamente incompleto su una parte significativa dei 111 DOI ristretti.

4. Prerequisiti (da verificare PRIMA di iniziare)

Nessuno di questi punti è formale: ognuno corrisponde a un incidente già occorso.

4.1 Sincronizzazione Dropbox disattivata sulla cartella di lavoro. Per il pilota è sufficiente la sospensione della sincronizzazione. Per il lotto completo va valutata la Selective Sync o lo spostamento della cartella fuori da Dropbox (vedi sezione 9).

4.2 Mount drvfs con opzione metadata. Senza di essa `utime()` fallisce e lo store dell'AIP termina con errore `rsync` codice 23 anche quando il trasferimento dei file è riuscito. Verifica:

```
mount | grep drvfs
# atteso: ... type drvfs (rw,noatime,...,metadata,uid=...,gid=...,umask=022)
cat /etc/wsl.conf
```

4.3 Spazio disponibile. Per il pilota bastano pochi MB, ma è il momento giusto per misurare l'headroom in vista del lotto (63 GB di download più lavorazione Archivematica e AIP risultanti):

```
df -h /mnt/e
```

4.4 File .env completo, con `DATAVERSE_API_KEY` valorizzata: **indispensabile** per il Pilota B (file ristretti). Verifica rapida senza stampare la chiave:

```
cd tools/dataverse-archivematica
grep -c DATAVERSE_API_KEY .env # atteso: 1
```

4.5 Utente di esecuzione. Gli script della pipeline **non vanno mai eseguiti come utente archivematica**. Verifica con `whoami`.

4.6 Versione degli script. Il pilota ha senso solo con la pipeline aggiornata:

```
git log --oneline -1
grep -c "_is_ingested_tabular" scarica_dataverse.py # atteso: >= 2
grep -c "doi_path.resolve()" archivematica_ingest.py # atteso: 1
```

4.7 Processing configuration corretta — preconditione critica. Due scelte di `dataverse_001` determinano la qualità archivistica degli AIP:

Decisione	Valore richiesto
Perform file format identification (Transfer)	Yes
Perform file format identification (Ingest)	<i>No, use existing data</i>
Normalize	Normalize for preservation
Approve normalization	Yes

Senza identificazione in fase di transfer, il METS riporta **Unknown** per **tutti** gli oggetti: niente PUID, niente pianificazione della conservazione, e la normalizzazione non può funzionare (il FPR sceglie le regole in base al formato riconosciuto). L'impostazione "*No, use existing data*" in ingest è corretta solo se l'identificazione in transfer è attiva, altrimenti non esistono dati da riusare.

Il file **non va modificato a mano**: gli UUID delle catene variano tra versioni di Archivematica. Si genera dal Dashboard (*Administration -> Processing configuration -> dataverse_001 -> Edit -> Save -> Download*) e si copia nella posizione della copia locale versionata. Verifica:

```
CFG=~/.arkive/config/dataverse_001ProcessingMCP.xml
grep -A2 "f09847c2-ee51-429a-9478-a860477f6b8d" $CFG # identificazione: NON "Skip"
grep -A2 "cb8e5706-e73f-472f-ad9b-d1236af8095f" $CFG # normalizzazione
grep -A2 "7509e7dc-1e1b-4dce-8d21-e130515fce73" $CFG # normalizzazione (2° punto)
```

5. Esecuzione

5.1 Camera pulita — DENTRO la Transfer Source Location

La ri-acquisizione **non** si sovrappone a cartelle esistenti: la pipeline salva ora i nomi originali (`.csv`, `.dta`), quindi su una cartella popolata in passato scaricherebbe gli originali **accanto** ai vecchi `.tab`, lasciando entrambi.

La cartella di lavoro deve inoltre trovarsi **fisicamente dentro il percorso della Transfer Source Location** dello Storage Service. Motivo: `archivematica_ingest.py` avvia il transfer passando `base64(<location_uuid>:<path_a` e lo Storage Service accetta solo percorsi interni alla location. Un pacchetto fuori da lì non verrebbe visto da Archivematica.

Allo stesso tempo la cartella deve essere una **sottocartella dedicata**, non la radice della Transfer Source: se si omette `--source`, lo script raccoglie *tutti* i pacchetti DOI presenti nella location, compresi quelli della vecchia acquisizione, che il pilota non deve toccare.

```
# 1) ricavare il percorso della Transfer Source Location (o usare quello noto)
TS_PATH=/percorso/della/transfer_source # adattare

# 2) creare la sottocartella dedicata al pilota, vuota
mkdir -p "$TS_PATH/PILOTA"
ls -A "$TS_PATH/PILOTA" # deve essere vuota
```

Se il download è già stato eseguito altrove, spostare i pacchetti (non copiarli, per non lasciare duplicati):

```
mv PILOTA/doi_* "$TS_PATH/PILOTA"/
```

5.2 Lista DOI del pilota

```
cat > dois_pilota.txt << 'EOF'
doi:10.13130/RD_UNIMI/PKZ7FJ
doi:10.13130/RD_UNIMI/DP08SC
EOF
```

5.3 Download

```
python3 scarica_dataverse.py --dois dois_pilota.txt --output "$TS_PATH/PILOTA" --dcat \
2>&1 | tee logs/pilota_download.log
```

--dcat è necessario: il pilota deve validare anche che la dcat:distribution descriva l'originale.

6. Gate 1 — Validazione dell'acquisizione

Non procedere all'ingest se anche uno solo di questi controlli fallisce.

6.1 Nessun errore di download. Nel riepilogo_download.json entrambi i DOI devono risultare ok, mai partial:

```
python3 -c "
import json,os; TS=os.environ['TS_PATH']
d=json.load(open(TS+'/PILOTA/riepilogo_download.json'))
print(json.dumps(d, indent=2)[:800])
"
grep -c "ERRORE\\|[RISCARICO]" logs/pilota_download.log    # atteso: 0
```

6.2 Nessun fallback alla derivata. Se compare [FALLBACK], per quel file format=original non ha risposto e nel pacchetto è finita la .tab:

```
grep "\\[FALLBACK\\]" logs/pilota_download.log    # atteso: nessun risultato
```

Controllo di contabilità speculare, che conferma il caso positivo:

```
grep -c "Scarico:" logs/pilota_download.log    # atteso: 36 (10 + 26)
grep -c "originale di" logs/pilota_download.log    # atteso: 31 (8 + 23)
```

Il primo è il totale dei file scaricati, il secondo quelli scaricati **come originale** al posto della derivata (etichetta nome.csv (originale di nome.tab)). La differenza (36 – 31 = 5) deve corrispondere ai file **non** ingeriti, cioè privi di un originale distinto. Se la contabilità non chiude, qualche file ingerito non è stato trattato come tale e va indagato prima di procedere.

6.3 Sono stati conservati gli ORIGINALI, non i .tab. È il controllo centrale del Livello 2:

```
find "$TS_PATH/PILOTA" -name "*.tab" | head    # atteso: NESSUN risultato
find "$TS_PATH/PILOTA" -path "*/objects/*" -type f -not -path "*/metadata/*" | head -20
```

I nomi vanno preservati esattamente come depositati. Nell'elenco è normale — e corretto — trovare nomi con spazi (IVD_NMBU shipment 2_cytotoxicity_NRU.xlsx) o con estensioni anomale ripetute (..._ANPEP.xlsx.xlsx). La pipeline usa originalFileName così com'è: quei nomi sono letteralmente quelli con cui il ricercatore ha depositato i file. Conservarli invariati, refusi compresi, è il comportamento corretto ai fini dell'autenticità dell'oggetto archiviato; una normalizzazione silenziosa produrrebbe oggetti diversi dagli originali.

6.4 Struttura per versione completa. Il Pilota A deve avere 4 versioni, il Pilota B 3:

```
ls -d "$TS_PATH"/PILOTA/*PKZ7FJ*/objects/*/ # atteso: 4 directory
ls -d "$TS_PATH"/PILOTA/*DP08SC*/objects/*/ # atteso: 3 directory
ls "$TS_PATH"/PILOTA/*DP08SC*/objects/*/metadata/
```

Asimmetria attesa nei metadati di versione: `dataverse.json`, `dcat.json` e `metadata.csv` sono presenti in **ogni** versione perché generati localmente dalla pipeline; `schema.json` compare **solo nell'ultima versione**, perché è scaricato dall'exporter Schema.org di Dataverse, che lavora a livello di dataset e non espone le versioni storiche. La sua assenza nelle versioni precedenti **non è un errore di export**. Da verificare invece se mancasse in modo irregolare (per esempio assente anche nell'ultima versione, o presente a intermittenza).

6.5 I file ristretti sono stati scaricati (Pilota B). Contro il rischio del pacchetto silenziosamente incompleto: il numero di file di dati presenti su disco deve corrispondere a quello atteso dal report di Fase 0.

```
find "$TS_PATH"/PILOTA/*DP08SC* -path "*/objects/*" -type f \
-not -path "*/metadata/*" | wc -l # atteso: 26
```

L'esclusione di `*/metadata/*` è necessaria: nella struttura `objects/<versione>/metadata/` la parola `objects` compare comunque nel percorso, quindi senza il filtro verrebbero conteggiati anche `dcat.json`, `metadata.csv`, `dataverse.json` e `schema.json`, falsando il totale.

6.6 Il DCAT descrive l'originale.

```
python3 -c "
import json,glob,os
TS=os.environ['TS_PATH']
for p in sorted(glob.glob(TS+'PILOTA/*/objects/*/metadata/dcat.json'))[:2]:
    d=json.load(open(p)); dd=d.get('dcat:distribution')
    dd=dd if isinstance(dd,list) else [dd]
    print(p)
    for x in dd[:3]:
        print(' ', x.get('dct:title'), '|', x.get('dcat:mediaType'), '|',
              x.get('dcat:accessURL',{}).get('@id','')[-30:])
"
```

Atteso: titoli con estensione originale (`.csv`, `.xlsx`, `.dta`), `mediaType` dell'originale (**non** `text/tab-separated-values`) e `accessURL` che termina con `?format=original`.

Attenzione — è normale che alcune `accessURL` NON abbiano `?format=original`. La pipeline decide file per file: i file **non ingeriti** da Dataverse (privi dei campi `original*`) non hanno una rappresentazione originale distinta, quindi il loro `accessURL` è quello semplice. Aggiungere `?format=original` significherebbe descrivere una rappresentazione inesistente. Un `.xlsx` può benissimo non essere stato ingerito (fogli multipli, dimensione, ingest non riuscito): non è un errore.

Verifica di contabilità sull'intero pilota, che chiude il cerchio con il log:

```
python3 - << 'EOF'
import json, glob, os
TS=os.environ['TS_PATH']
tot=orig=plain=0
for p in sorted(glob.glob(TS+'PILOTA/*/objects/*/metadata/dcat.json')):
    d=json.load(open(p)); dd=d.get('dcat:distribution') or []
    dd=dd if isinstance(dd,list) else [dd]
    for x in dd:
        tot+=1
        u=x.get('dcat:accessURL',{}).get('@id','')
        if u.endswith('?format=original'): orig+=1
        else: plain+=1
        if 'tab-separated' in (x.get('dcat:mediaType') or ''):
            print('!! ANOMALIA (mediaType .tab):', p, x.get('dct:title'))
```

```
print(f"distribution totali: {tot} con format=original: {orig} senza: {plain}")
EOF
```

Atteso: **36 totali, 31 con format=original, 5 senza** — gli stessi numeri dei `grep` del punto 6.2. Nessuna riga di anomalia. In caso di dubbio su un singolo file, la conferma si legge in `dataverse.json`: se `originalFileName` e `originalFileFormat` sono `None`, il file non è ingerito e l'URL semplice è corretto.

6.7 metadata.csv coerente. La colonna `filename` è costruita leggendo il disco, quindi deve elencare i nomi originali:

```
head -3 "$TS_PATH"/PILOTA/*PKZ7FJ*/metadata/metadata.csv
```

7. Ingest e Gate 2 — Validazione dell'AIP

7.1 Ingest

```
python3 archivematica_ingest.py --source "$TS_PATH/PILOTA" --auto-approve \
--state-file stato_pilota.json 2>&1 | tee logs/pilota_ingest.log
```

Due accorgimenti, entrambi necessari:

- **--source esplicito.** Omettendolo, lo script userebbe la radice della Transfer Source Location e raccoglierebbe *tutti* i pacchetti DOI presenti, inclusi quelli della vecchia acquisizione: il pilota ingerirebbe molto più del previsto. Puntando alla sottocartella si lavora solo sui due dataset.
- **File di stato separato** (`stato_pilota.json`), per non mescolare il pilota con lo storico di produzione in `stato_archivematica.json`.

Prima di lanciare, verificare che nella sottocartella ci siano esattamente i due pacchetti attesi:

```
ls -d "$TS_PATH/PILOTA"/doi_*          # atteso: 2 directory
```

7.2 Gate 2 — controlli sull'AIP

7.2.1 AIP presente e memorizzato. Annotare gli UUID e verificare lo stato:

```
python3 - << 'EOF'
import os, requests, urllib3
urllib3.disable_warnings()
for l in open('.env'):
    l=l.strip()
    if l and not l.startswith('#') and '=' in l:
        k,_,v=l.partition('='); os.environ.setdefault(k.strip(), v.strip().strip('\'))
SS=os.environ.get('SS_URL','http://localhost:8000')
H={'Authorization': f"ApiKey {os.environ['SS_USER']}:{os.environ['SS_API_KEY']}"}
for nome,uuid in [( 'DP08SC', '<AIP-UUID>' ), ( 'PKZ7FJ', '<AIP-UUID>' )]:
    d=requests.get(f"{SS}/api/v2/file/{uuid}/", headers=H, verify=False, timeout=60).json()
    print(f"{nome}: status={d.get('status')} size={d.get('size')}")
    print(f"    {d.get('current_full_path')}")
EOF
```

Atteso: `status=UPLOADED`. Le dimensioni devono essere coerenti con il volume degli originali stimato in Fase 0.

7.2.2 Formati, eventi e agenti — il controllo decisivo. Legge il METS dall'AIP compresso senza estrarlo:

```
python3 - << 'EOF'
import tarfile, glob, re
from collections import Counter
import xml.etree.ElementTree as ET
```

```

for uuid, nome in [(('<primi8>', 'DP08SC'), ('<primi8>', 'PKZ7FJ'))]:
    aip = glob.glob(f"/mnt/e/ARKIVE/AIPsStore/{uuid[:4]}/**/*{uuid}*.tar.gz", recursive=True)
    with tarfile.open(aip[0]) as t:
        mets=[m for m in t.getnames() if re.search(r'/data/METS\..*\.xml$', m)]
        xml=t.extractfile(mets[0]).read()
        root=ET.fromstring(xml); tag=lambda e: e.tag.split(' ')[-1]
        print(f"\n=== {nome}")
        for f,n in Counter(e.text for e in root.iter() if tag(e)=='formatName').most_common():
            print(f"    {n:4d}    {f}")
        for e,n in Counter(x.text for x in root.iter() if tag(x)=='eventType').most_common():
            print(f"    [ev] {n:4d}    {e}")
EOF

```

Attesi e criteri di lettura:

- i formati devono essere **identificati**: Microsoft Excel, CSV, Plain Text, JSON Data Interchange Format... e **nessun Unknown**;
- **nessun Tab-Separated Values** tra gli oggetti: se comparisse, nell'AIP è finita una derivata invece dell'originale;
- deve esserci un evento **format identification** per ogni oggetto;
- gli eventi **normalization** compaiono **solo per i formati con una regola FPR** (tipicamente PDF, immagini, audio, video). La loro assenza sugli .xlsx non è un guasto: il FPR considera quei formati già adeguati alla conservazione;
- l'agente e la versione dello strumento di identificazione si leggono nell'**eventDetail** (es. `program="Siegfried"; version="1.11.2"`).

7.2.3 Conteggio degli oggetti — attenzione a cosa si conta. Il numero di file nel METS è **maggiore** di quello dei file di dati, perché i metadati per versione (`objects/vX.Y/metadata/...`) stanno dentro `objects/` e Archivematica li tratta come contenuto. Per DP08SC: 26 file di dati + 10 metadati = 36; per PKZ7FJ: 10 + 13 = 23. Solo la cartella `metadata/` di primo livello è trattata come metadati descrittivi.

7.2.4 Nomi originali preservati. Archivematica **sanifica** i nomi con spazi o caratteri problematici sul filesystem e registra ogni modifica con un evento **filename change**. Il nome di deposito resta documentato nel METS in `originalName`. Verifica che gli spazi ci siano ancora:

```

python3 - << 'EOF'
import tarfile, glob, re
import xml.etree.ElementTree as ET
uuid='<primi8>'
aip=glob.glob(f"/mnt/e/ARKIVE/AIPsStore/{uuid[:4]}/**/*{uuid}*.tar.gz", recursive=True)
with tarfile.open(aip[0]) as t:
    mets=[m for m in t.getnames() if re.search(r'/data/METS\..*\.xml$', m)]
    xml=t.extractfile(mets[0]).read()
    root=ET.fromstring(xml); tag=lambda e: e.tag.split(' ')[-1]
    orig=[e.text for e in root.iter() if tag(e)=='originalName' and e.text and not e.text.endswith('/')]
    print("con spazi nel nome:", sum(1 for o in orig if ' ' in o.split('/')[-1]), "su", len(orig))
EOF

```

7.2.5 Fixity. Nessun evento PREMIS di *fixity check* fallito.

8. I 6 DOI non più raggiungibili

QA4BR0, BX8AIN, YF2NZV, WKMYTV, YIAURF, OYDM83 non esistono più su `dataverse.unimi.it` (verificato via API, browser e `curl`). Per questi la ri-acquisizione è **impossibile**, quindi:

1. **Prima di qualunque pulizia**, verificare se ne esiste una copia locale (nelle cartelle di download o negli AIP già prodotti) e, in caso affermativo, **metterla al sicuro fuori dall'area di lavoro**: sarebbe l'unico esemplare rimasto.

2. **Segnalare la scomparsa ai responsabili di Data@UNIMI.** Sei dataset pubblicati con DOI assegnato che spariscono da un repository certificato CoreTrustSeal richiedono una spiegazione (deaccession su richiesta dell'autore, DOI di test rimossi, o altro), che va documentata.
3. Non includerli nella lista del lotto.

9. Criteri di GO / NO-GO per il lotto completo

Si procede al lotto dei 702 dataset **solo se**:

- entrambi i Gate del pilota sono superati senza eccezioni;
- in particolare il Pilota B dimostra che i file **ristretti e ingeriti** vengono scaricati come originali (nessun [FALLBACK], conteggio file corretto);
- è stata presa una decisione **strutturale** sulla collocazione di TRANSFER_SOURCE: 63 GB in 67 730 file dentro una cartella sincronizzata con Dropbox riproporrebbero i lock transitori su larga scala. La sospensione manuale della sincronizzazione non è sufficiente per una lavorazione lunga, perché Dropbox può riprendere dopo un riavvio o un aggiornamento del client;
- lo spazio disponibile copre 63 GB di download più lavorazione e AIP;
- è stato definito il **dimensionamento dei lotti** (per esempio 50–100 DOI per volta) e il punto di ripresa in caso di interruzione. I lotti si organizzano in **sottocartelle della Transfer Source Location** (LOTTO_01, LOTTO_02...), da indicare con --source: è verificato che Archivematica le risolva correttamente. Omettere --source farebbe usare la radice della location, con il rischio di rilavorare tutti i pacchetti presenti;
- la **processing configuration** è quella corretta (prerequisito 4.7): un errore qui obbligherebbe a rifare tutti gli AIP del lotto.

Nota a favore: la ri-acquisizione è ripartibile in sicurezza. Lo skip si basa su dimensione e checksum, quindi rilanciare non lascia file monchi; un download interrotto viene ri-caricato al passaggio successivo.

10. Registrazione dell'esito

Al termine del pilota annotare, per ciascun dataset: data di esecuzione, esito dei due Gate, UUID dell'AIP prodotto, anomalie riscontrate e correzioni applicate. Questo verbale è la base documentale della ricostruzione del corpus e va conservato insieme alla documentazione del progetto.

10.1 Esito del pilota del 22 luglio 2026

Gate 1 superato su entrambi i dataset: 36 file scaricati (10 + 26), di cui 31 come originali (format=original) e 5 non ingeriti; nessun [FALLBACK], nessun .tab su disco, nessun errore o riscalco. Le 4 versioni di PKZ7FJ e le 3 di DP08SC complete. Le 36 dcat:distribution coerenti: 31 con ?format=original, 5 senza (file non ingeriti).

Gate 2 superato: formati identificati da Siegfried 1.11.2 (25 Microsoft Excel su DP08SC, più PDF, CSV, JSON, Plain Text), nessun Unknown, nessun Tab-Separated Values. Su PKZ7FJ 2 eventi di normalization con 2 derivate PDF/A validate. I 25 nomi con spazi risultano sanificati sul filesystem e documentati in originalName con relativi eventi filename change.

Anomalie riscontrate e risolte durante il pilota:

1. *Ingest fallito con i pacchetti in sottocartella* (Filepath ... does not exist): bug di archivematica_ingest.py, che ricostruiva il percorso dal solo nome della cartella perdendo la sottocartella. Corretto passando il path assoluto del pacchetto; verificato con un transfer completato da sottocartella.
2. *Tutti i formati Unknown al primo ingest:* identificazione dei formati disattivata nella processing configuration (vedi prerequisito 4.7). Corretta e ingest ripetuto.
3. *Permessi in scrittura sulla Transfer Source:* il mount drvfs usa uid=333;gid=333;umask=022, quindi al gruppo manca il bit w. Risolto con chmod g+ws sulla cartella; per il lotto va valutata una soluzione stabile (umask=002 in /etc/wsl.conf o riposizionamento della location).